Available online at www.sciencedirect.com

ScienceDirect

Procedia CIRP 00 (2026) 000–000

www.elsevier.com/locate/procedia

14th CIRP Global Web Conference (CIRPe 2026)

Comparison of CAD Model Representations for Machine Learning: Guidance for Representation Selection

Daniel Mansfeldt[a,b,*], Michel Kruse[a,b], Lasse Päplow[a,b]

[a] *F&E GmbH, Schwentinestr. 24, 24149 Kiel, Germany*
[b] *Institut für Produktionstechnik und CIMTT, Schwentinestr. 13, 24149 Kiel, Germany*

* Corresponding author. *E-mail address:* daniel.mansfeldt@haw-kiel.de

Abstract

Selecting an appropriate representation for CAD data processing remains a fundamental challenge, as it requires balancing geometric accuracy with computational efficiency for downstream tasks. While representations such as voxels, point clouds, and meshes have been extensively studied in the context of 3D machine learning, representations such as dimensionless invariants, STEP file-based trees, and vector sets represent comparatively recent approaches, each rooted in a distinct and widely adopted paradigm — descriptors, graphs, and neural fields, respectively. This paper presents a structured comparison of these approaches, including multi-view images as a strong baseline, evaluating them on classification and regression tasks using the FabWave dataset. The authors aim to characterize the strengths and limitations of each approach, providing actionable guidance on selecting a representation suited to specific tasks.

Keywords: CAD; 3D Model Representation; Machine Learning; Classification; Regression; Comparison

1. Introduction

Add link to the code repository?

A wide variety of machine learning (ML) architectures have been developed to handle the many possible representations of geometric data. Each representation requires its custom ML architecture to be processible in the first place. This means that a variety of representations and their corresponding model layout must be compared when developing a new application to determine the most suitable one.

Geometric data can be conveyed using a variety of representations. These include multi-view images [2], voxels [3], [9], meshes

referenz einfügen

(quelle?) and graphs [4], among others. The different representations of essentially the same information require the use of a variety of ML-based approaches for CAD data analysis.

Unfortunately, each representation has its own advantages and disadvantages. Voxels for instance limit the maximum resolution of CAD models due to a cubic increase in voxel number and limited computing resources. This makes it challenging to choose the best representation form and associated ML model for a given task. To help guide the decision on which data representation form to use, we compare four promising machine learning approaches, each using a distinct 3D data representation. By comparing these approaches on general CAD attributes - i.e., not specific to most production use cases - we show which ML model, along with its input data representation, can be expected to perform well across many applications.

All ML models will be trained and tested on the same data sets. As such, we used the Fabwave dataset [8] and preprocessed its data for comparison among the different approaches.

Our contributions include:

- Comparison of four different CAD model representation forms including; dimensionless invariants, trees based on STEP files, multi-view images and vector sets.
- Performance comparison of data representation and ML model combinations in terms of classification of real engineering parts.
- Performance comparison of data representation and ml model combinations in terms of regression tasks aim-

ing at predicting basic properties of genuine engineering parts.

The Rest of this paper is structured as follows. In section 2, we review related work. This includes the foundations of the ML models examined and the CAD model representation types used. Section 3 presents our methodology for prepareing the data set and comparing model performance. Section 4 present our results and their discuss their implication. Finally, section 5 provides a conclusion and lists possible future works.

2. Related Work

As we aim to determine the best pair of CAD model representation and ML architecture, we lay out the different representation schemes and ML architectures considered.

2.1. Representation of 3D CAD Models for Processing via Machine Learning Models

Research on 3D data representations for machine learning has explored a wide range of approaches. These include point clouds, pure primitive instancing, cell decomposition (meshes), constructive solid geometry, sweep representations and boundary representations (B-Rep) [7]. While each of these representation schemes serves certain use cases, they are not necessarily directly processable by a given machine learning model. To address this issue multiple techniques have also been developed to derive a data structure from the CAD-model that conveys the topological and geometric information in a way that machine learning models can process. For this work four such techniques were utilized. Each one will be laid out in detail next. To the best of our knowledge these representation techniques of 3D data have not been compared to each other in terms of a geometry analysis use case.

2.1.1. Images

CAD models can be converted into image sets for processing by machine learning models. Given the widespread use of convolutional neural networks for image analysis, this represents a natural and accessible approach.

However, adequate representation of a CAD model requires images from a sufficiently large number of viewpoints. If the number of views is too limited, relevant geometric features may not be captured. This challenge is particularly pronounced for internal features, which cannot be observed from external viewpoints alone. Although this limitation can be mitigated by incorporating cross-sectional images taken at multiple slice positions, doing so significantly increases the number of required images and, consequently, the computational cost.Therefore, an appropriate balance must be found between representational fidelity and computational efficiency.

Several prior studies have employed image-based approaches for CAD model analysis [2].

weitere quellen hinzufügen.

2.1.2. Tree Data Structure

Miles et al. emphasize that B-Rep models possess an inherent hierarchical structure. For instance, a line is defined by a point and a direction vector. Based on this property, topological entities such as vertices, edges, and faces can be modeled as a graph, with geometric information incorporated through coordinate-based attributes. Such a representation is well suited for the application of graph neural networks to CAD models. [4]

2.1.3. Dimensionless Invariants

Another approach to obtain an encoding vector was introduced by Kaiser et. al. based on the Pi-Theorem

quelle fixen

They argue that any three-dimensional body can be described by a set of dimensionless invariants. To derive them from a CAD model the model first is meshed and statistical moments on the mesh are calculated. Then the invariants are derived from these moments via the Buckingham Pi-theorem. These features are inherently invariant under translations but not to rotation. To establish a meaningful connection between feature vectors across different geometries, it is necessary to ensure uniform orientation and mesh normalization. With these regularized feature vectors, morphological similarity can be reduced to a distance metric but entails a loss of topological and absolute scale information. Additionally, the quality of the results is strongly influenced by the mesh quality. The automated meshing of large datasets with variable geometric characteristics represents a notable limitation of this approach. Nevertheless, the compact vectors offer a key advantage, enabling highly efficient storage and retrieval at scale. Combined with recent advances in mesh-generation algorithms, this suggests strong potential to further improve robustness.

2.1.4. Vecset

Recent advances in 3D representation learning have explored a range of approaches. While classical methods rely on explicit representations such as meshes

welche Referenz genau?

[Garland and Heckbert 1997] or rasterized Structures like Voxel [9] or Octrees , the geometric neural field approach [5**?**] encodes shapes as continuous functions represented by the network itself. This leads to a high training effort for large amounts of data and to a lack of a regularized latent representation, due to the separately trained and thereby shape-specific networks. To bridge this gap, 3DShape2VecSet [10] introduces a hybrid approach that represents shapes as sets of latent vectors (VecSets), thereby decoupling shape-specific information into learnable vector sets while maintaining a single network across all shapes. It employs cross-attention mechanisms to query spatial coordinates, achieving a compact yet expressive shape encoding. This design enables efficient integration with generative models, particularly diffusion models

quelle einfüg

welche quelle genau?

[Ho et al. 2020], allowing for tasks such as unconditional generation, text-conditioned synthesis, and shape completion.

2.2. Architecture of Machine Learning Models for CAD Model Processing

Different CAD model representations require different machine learning architectures. This section introduces the architectures considered in this work.

2.2.1. Multi-layer Perceptron

Multi-layer perceptrons (MLPs), also referred to as feedforward neural networks, are among the most fundamental architectures in deep learning. For single-vector inputs, MLPs can be used for both classification and regression. In this work, they are therefore employed to make predictions from invariant representations and vector sets.

MLPs are also frequently integrated into more specialized machine-learning architectures. For example, they are commonly used as prediction heads, where they take the output vector of a preceding model component as input and produce the final prediction vector. In this capacity, MLPs are also used as components of the other architectures evaluated in this paper.

2.2.2. Child-Sum-Tree-LSTM

Miles et al. used a Child-Sum Tree-LSTM to generate encoding vectors for CAD models [4]. The model recursively traverses the tree structure of the CAD representation and computes, at each node, an encoding vector for the corresponding topological entity. This computation incorporates both the entity type represented by the node and the encodings of its child nodes. Through this recursive aggregation, the Tree-LSTM produces a single encoding vector for the entire CAD model, which can subsequently be used as input to a range of downstream machine-learning models.

2.2.3. Convolutional Neural Network

Convolutional neural networks (CNNs) are widely used machine-learning architectures for image-processing tasks. They are typically composed of convolutional and pooling layers, which extract local features from an image. These features are then commonly passed to a multi-layer perceptron (MLP), which produces the final prediction [1].

Pretrained models such as YOLO are widely used for object detection and image classification [6]. Whereas YOLO processes a single image as input, other architectures, such as RotationNet, process multiple images to generate a single prediction [2].

3. Methodology

3.1. Data Set

We use the FabWave dataset [8] to benchmark the proposed models. The dataset contains a broad range of CAD models, predominantly representing mechanical components. In its original form, it comprises more than 4,000 parts in STEP format assigned to 44 object categories.

Before the dataset could be used for model training and evaluation, extensive preprocessing was required. The individual preprocessing steps are described below.

First, the dataset was filtered. Some CAD models contain overlapping or disjoint bodies, which cannot be converted into all representations considered in this work. These models were therefore either fused into a single closed shell or removed from the dataset. In addition, object classes containing fewer than ten CAD models were excluded.

Second, the different representations described in section 2. were generated for each remaining CAD model. Should this have not been posible for all representations the CAD model was excluded from the data set entierly.

Third, the target values were derived for each CAD model. These targets include the part class, volume, number of faces, number of edges, and number of vertices.

Finally, the data splits were prepared. To reduce class imbalance, CAD models were oversampled such that each class contained at least half of the median class size. The dataset was then divided into training, validation, and test sets. Table 1 provides metadata for each split. Finally, the natural logarithm was applied to the numerical target values before training the machine-learning models.

Table 1. Data Splits: Metadata for the training, validation and test splits of the dataset.

Data Split	Fraction	Total Number of Samples
Training	0.6	1662
Validation	0.2	554
Test	0.2	554

3.2. Model Comparison

We compare the evaluated models separately for the classification and regression tasks.

In the classification task, the machine-learning models distinguish between 31 object classes. Ideally, abstract model categories would be used to assess the generalization capabilities of the models more broadly. However, defining complex abstract CAD models while still assigning them to meaningful and consistent classes is challenging. We therefore use the object classes provided by the FabWave dataset. Since the dataset primarily contains mechanical-engineering components, the resulting assessment is most directly applicable to CAD model analysis in the mechanical-engineering domain. Nevertheless, the results also provide insights into the applicability of the models to other 3D CAD analysis tasks, albeit to a lesser extent.

Classification performance is evaluated using accuracy, F1-score, recall, and precision. The latter three metrics are macro-averaged across all classes to account for class-level performance independently of class frequency. In addition, confusion matrices are provided to enable a more detailed comparison of the model's performance on individual classes.

The regression task aims to predict abstract and generally applicable properties of CAD models. The target quantities therefore include the volume and the numbers of vertices, edges, and faces. These properties can be computed for CAD models independently of their domain or specific use case and are thus suitable for evaluating the broader applicability of the models.

RotationNet is not considered for the regression task, as it was originally designed for classification. Although adapting RotationNet for regression appears feasible, such a modification is outside the scope of this work.

4. Results and Discussion

This section presents the results of the comparison between the different CAD analysis approaches. Classification and regression results are reported and interpreted separately. In addition, overarching conclusions are drawn to identify the most favorable approach across both tasks.

Please refer to the Appendix for all diagrams referenced in this section.

4.1. Classification

Figure .1 and table 2 summarize the overall classification performance. Across all evaluation metrics, the same ranking of approaches is observed. From best to worst, the approaches are ranked as follows: trees, images, vector sets, and invariants. Based on these results, the tree-based approach can be regarded as the dominant approach for the classification task.

Nevertheless, the other approaches may still achieve sufficiently high performance for practical application, depending on the specific requirements of a given use case. In particular, they may be preferable if other factors, such as computational efficiency, implementation complexity, or data availability, outweigh the observed loss in prediction quality.

add table with classifiaction metrics

Table 2. Classification Metrics: Listing of classification metric values as visualised in fig. .1. The highest value per metric is highlighted in bold font.

Model	Accuracy	F1-Score	Recall	Precision
Vecset	0	0	0	0
Trees	0	0	0	0
Images	0	0	0	0
Invariants	0	0	0	0

Figures .2 to .6 show the confusion matrices for all approaches. Each one manages to recognize most classes with accuracies close to one. Considering the overall classification metrics this was to be expected. Concurrently the image, invariant and tree approach each fails to recognize one class correctly entierely. The tree approach for instances confuses all bearings with washers. At the time of writing no hypothesis has been formed why this might be the case.

4.2. Regression

Figure .7 shows the distributions of the absolute errors for the regression target quantities relative to their target values. It should be noted that the displayed violin areas are affected by the logarithmic scale and are therefore not directly comparable in terms of absolute area.

As in the classification task, the tree-based approach achieves the best overall performance compared with the other approaches. This conclusion is also supported by the mean absolute error, mean squared error, and R2-score, which are shown in figs. .8 to .10.

5. Conclusion and Future Works

We compared three machine learning approaches for CAD model analysis. Each approach was evaluated on the FabWave dataset [8] using a classification task with high class diversity, as well as several abstract regression tasks.

The results show that the combination of tree-structured STEP file representations and a Tree-LSTM model, as proposed by Miles, achieved the best performance across all evaluated tasks. We therefore conclude that this method provides a promising starting point for more specialized CAD model analysis applications. Nevertheless, the other approaches produced comparable results in some cases and may also be suitable, particularly when considerations other than predictive performance favor them over the tree-based approach.

Future work should investigate the performance of the tree-based method on additional datasets. In particular, constructing datasets with more abstract learning tasks could help further demonstrate the model's general applicability.

The generalization capabilities of the evaluated methods could also be explored by incorporating additional features into the comparison, such as bounding-box dimensions, surface-to-volume ratio, or shape complexity.

Acknowledgements

decide on what to write here

Acknowledgements and Reference heading should be left justified, bold, with the first letter capitalized but have no numbers. Text below continues as normal.

Appendix

The appendix contains all diagramms referenced in section 4.

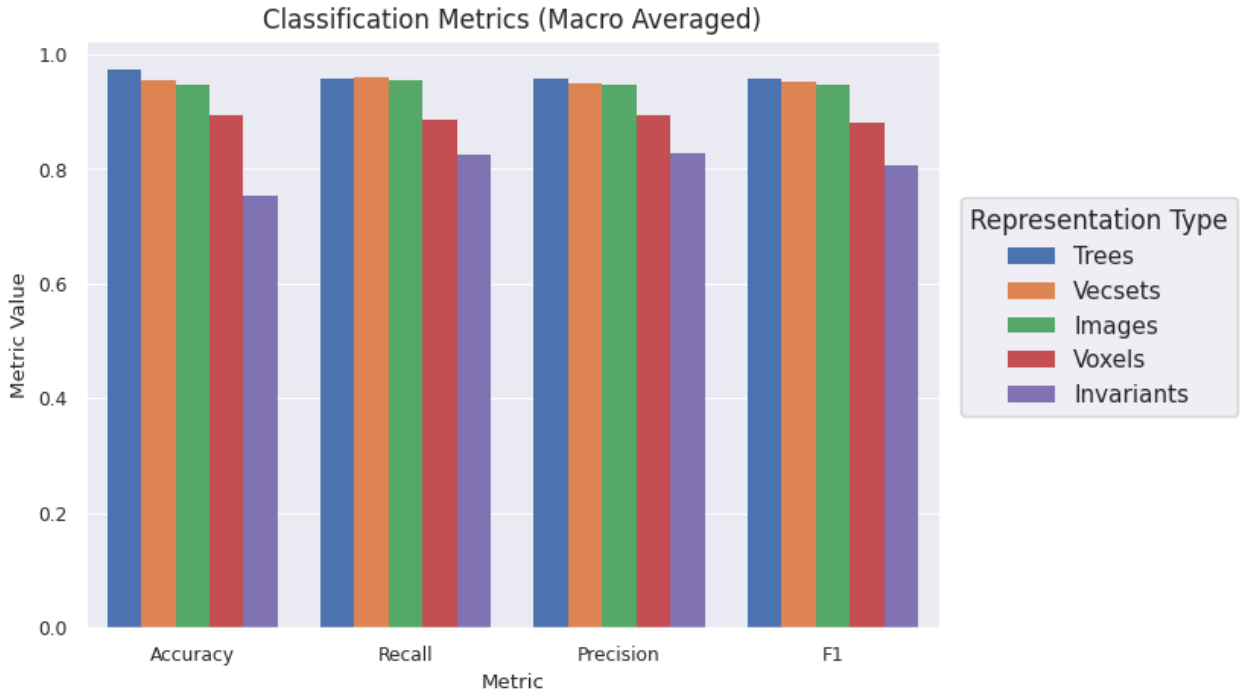

Fig. .1. Classification Metrics: Comparison of the different approaches in terms of overall Accuracy and macro-averaged F1-Score, Recall and Precision. The tree representation and associated tree-LSTM model is dominating the others across all metrics.

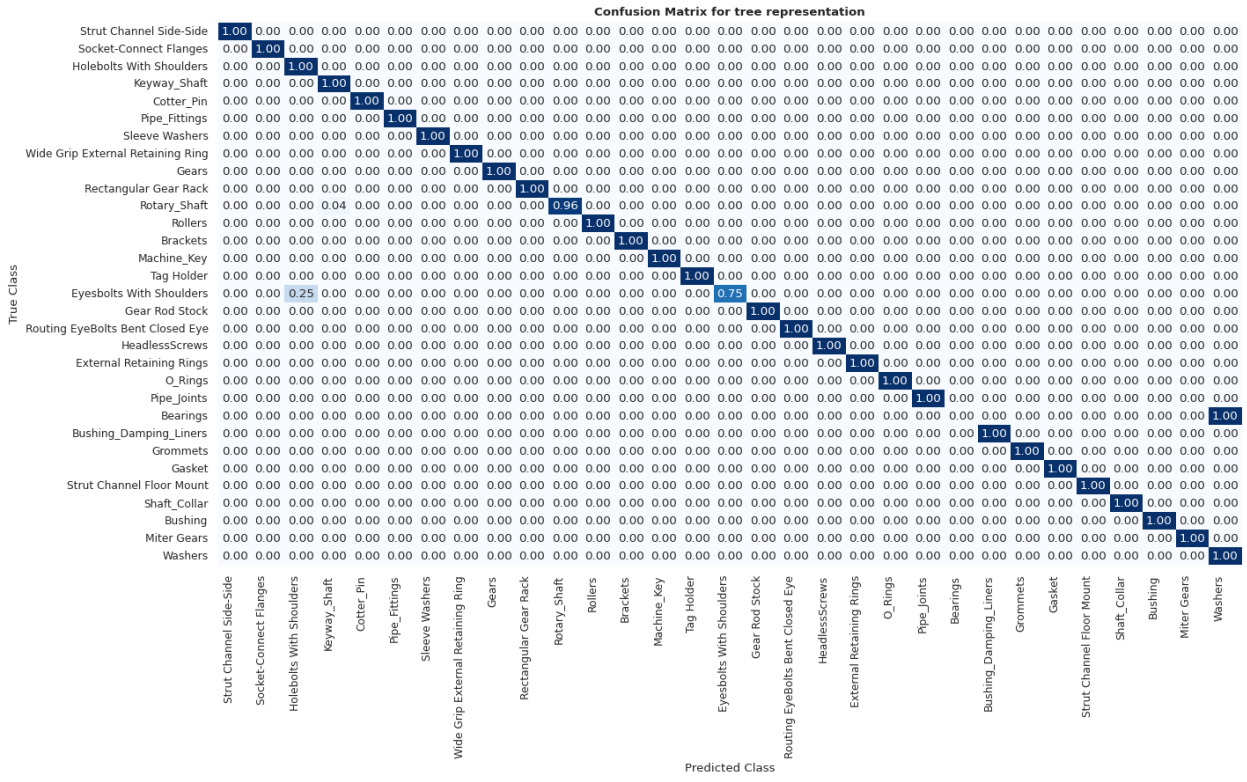

Fig. .2. Confusion Matrix for the tree-based approach.

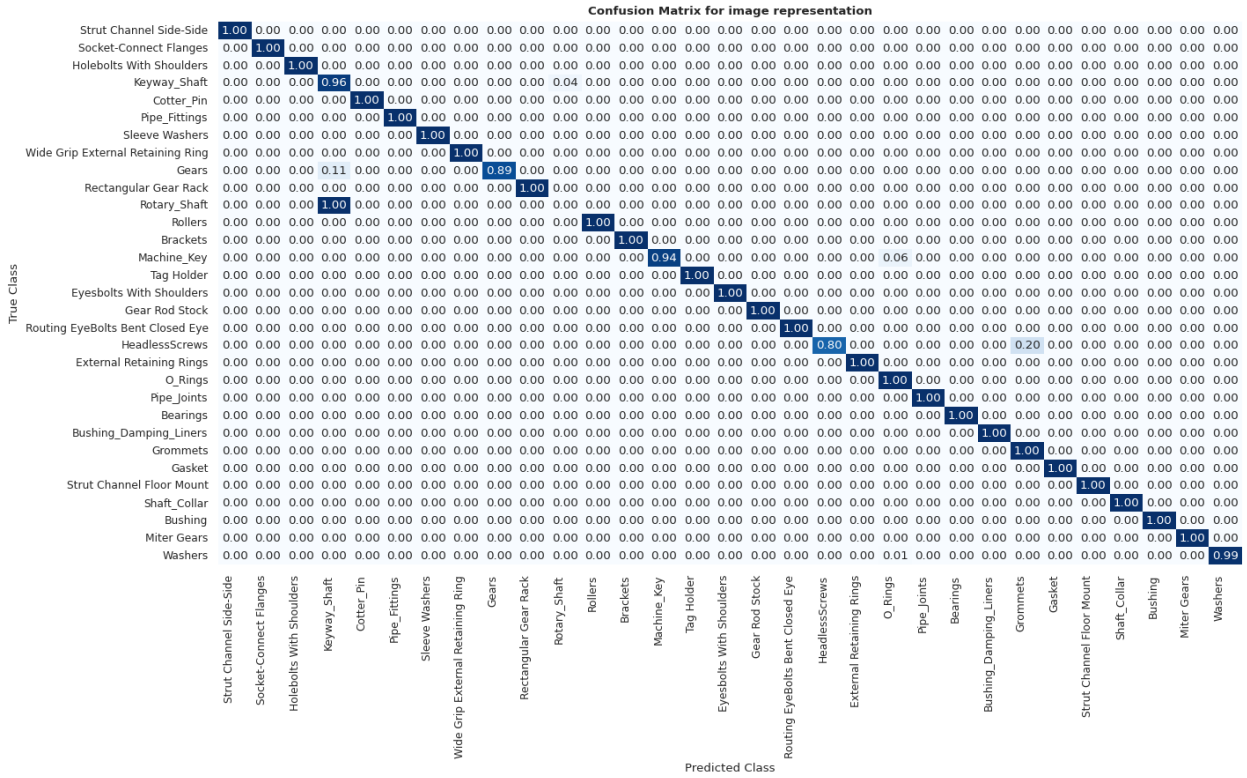

Fig. .3. Confusion Matrix for the image-based approach.

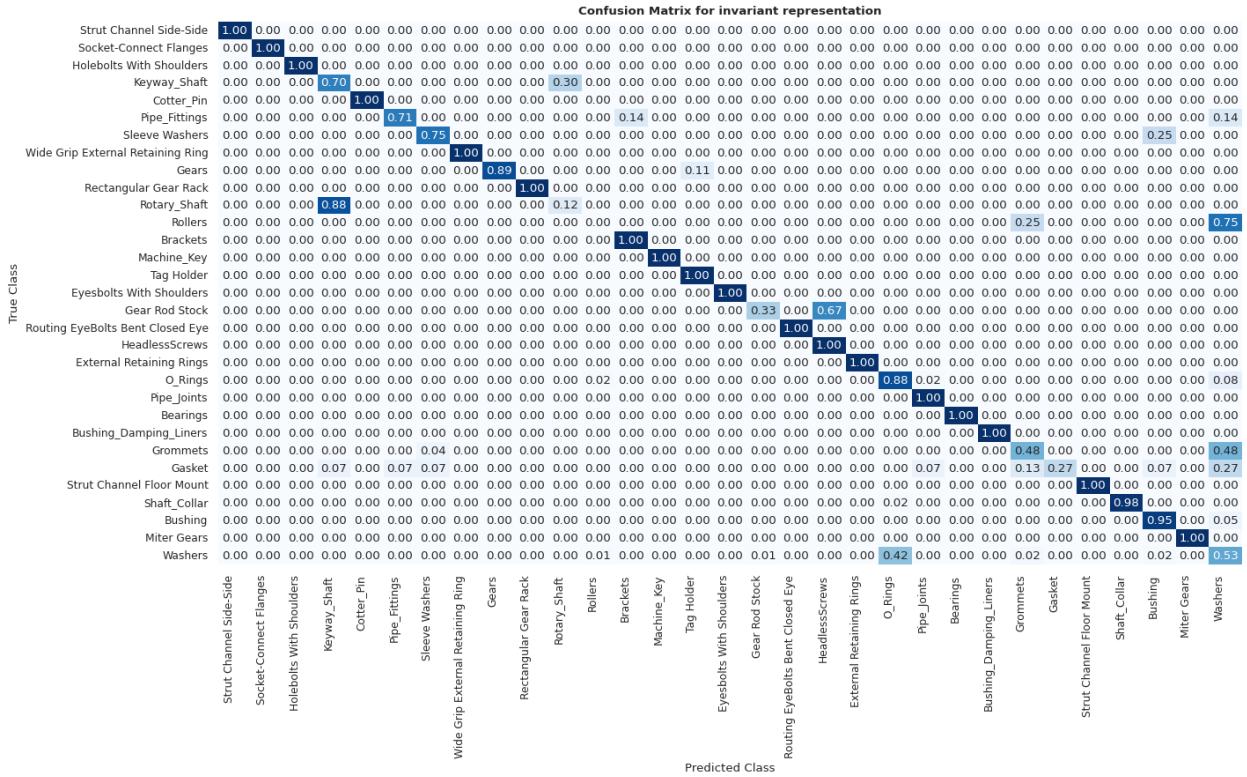

Fig. .4. Confusion Matrix for the invariant-based approach.

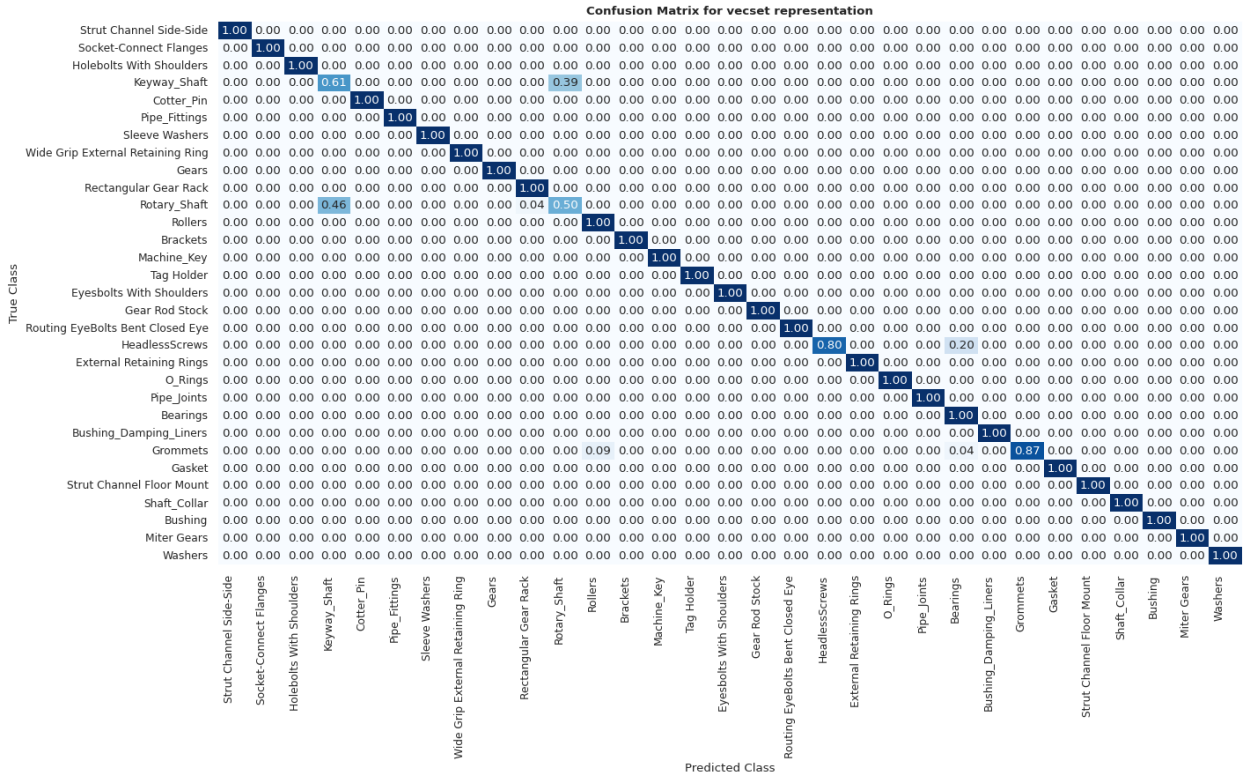

Fig. .5. Confusion Matrix for the vector-based approach.

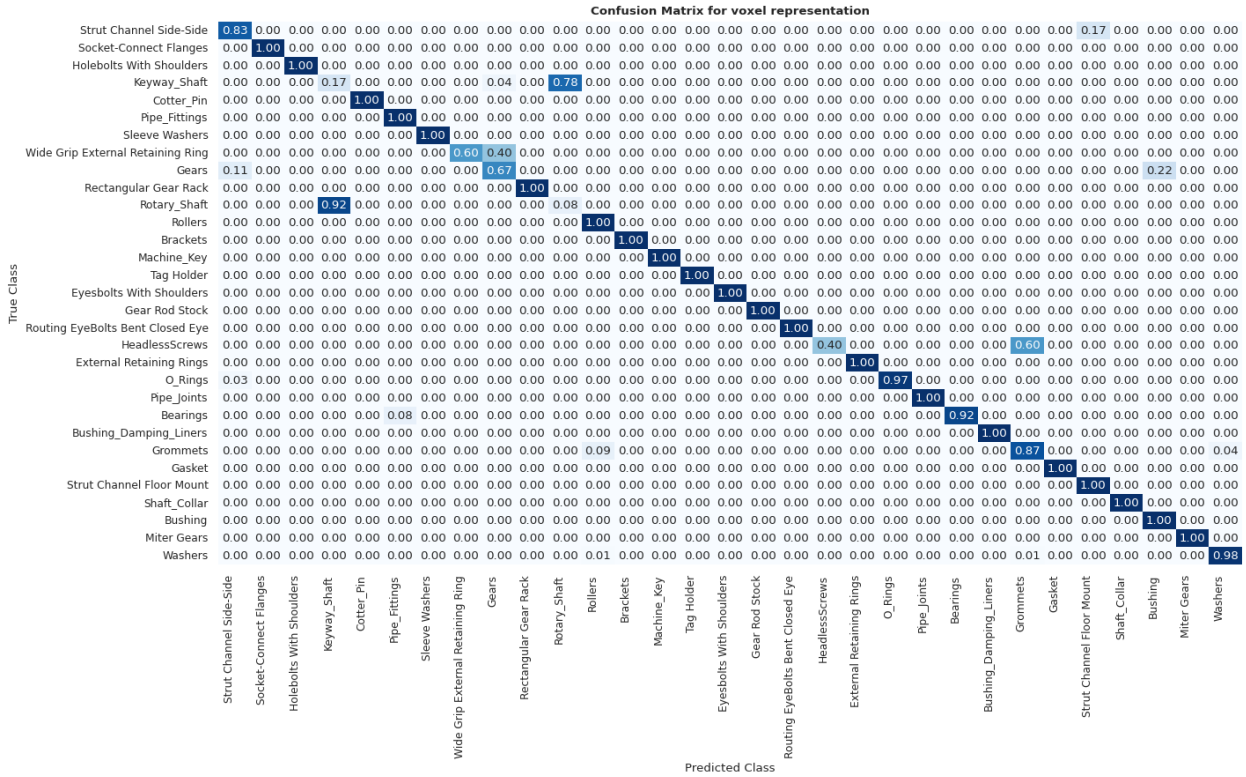

Fig. .6. Confusion Matrix for the voxel-based approach.

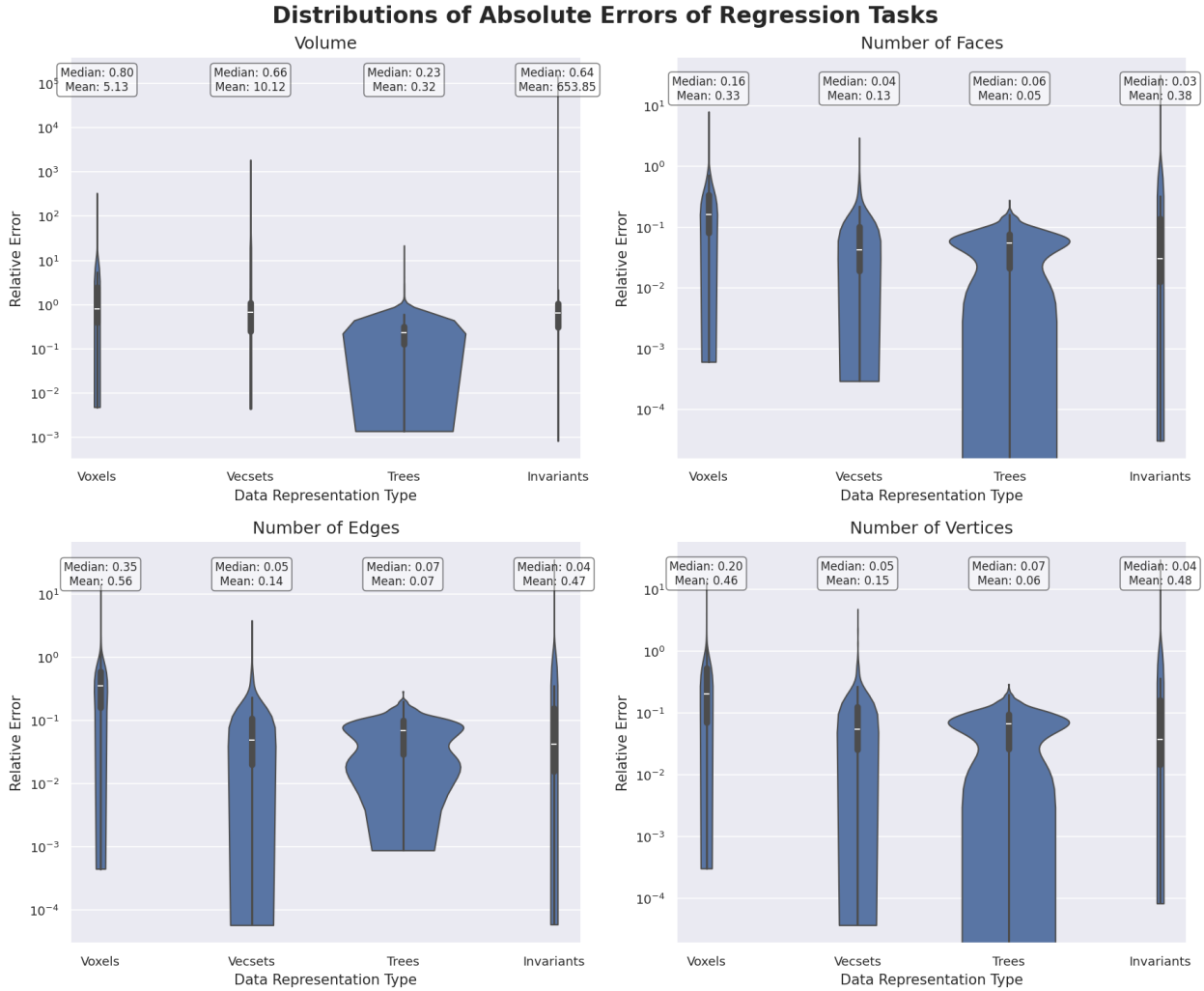

Fig. .7. Violinplot of Error Distributions: Each subplot shows the distributions of the relative absolute errors for one of the regressions features and for each approach considered. Median and mean values are provided for each distribution.

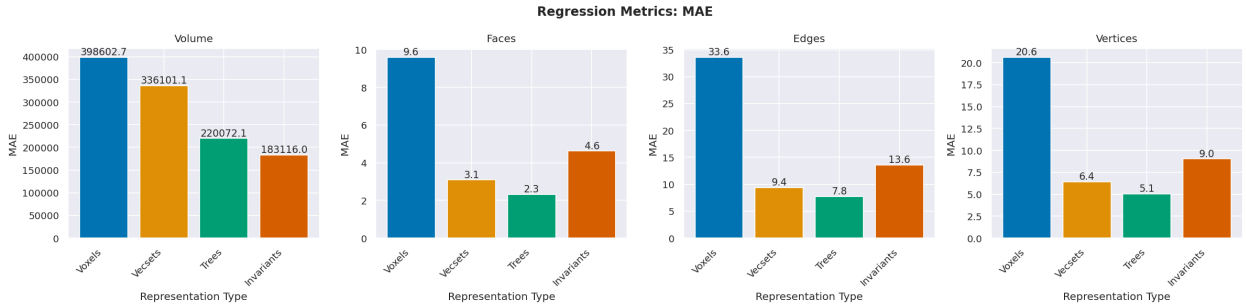

Fig. .8. Mean Absolute Errors: The plot shows the mean absolute errors for each regression target and approach.

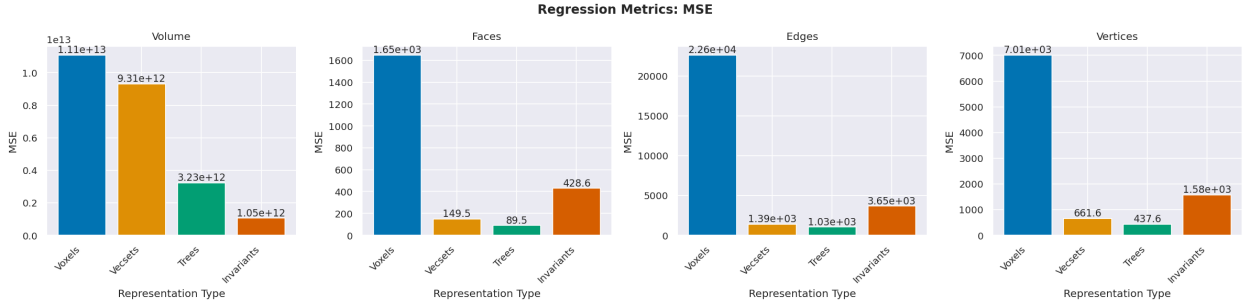

Fig. .9. Mean Squared Errors: The plot shows the mean squared errors for each regression target and approach.

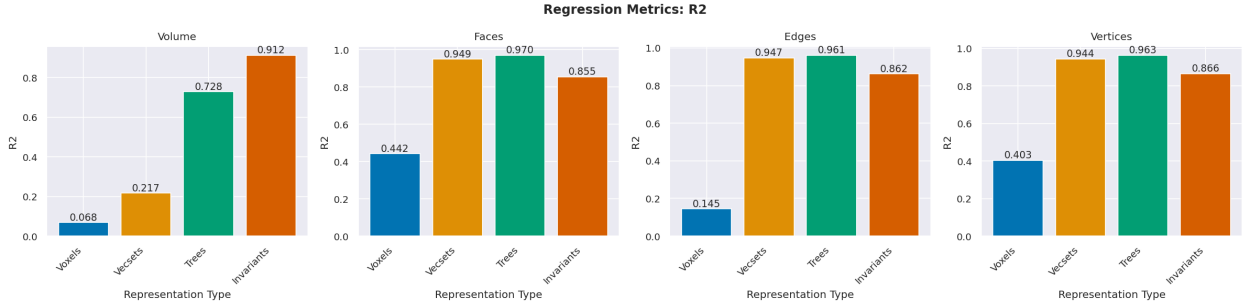

Fig. .10. R² Scores: The plot shows the R² scores for each regression target and approach.

References

[1] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning*, volume 112. Springer, 2013.

[2] Asako Kanezaki, Yasuyuki Matsushita, and Yoshifumi Nishida. Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5010–5019.

[3] Zhijian Liu, Haotian Tang, Yujun Lin, and Song Han. Point-voxel CNN for efficient 3d deep learning. _eprint: 1907.03739.

[4] Victoria Miles, Stefano Giani, and Oliver Vogt. Recursive encoder network for the automatic analysis of STEP files. 34(1):181–196.

[5] Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. _eprint: 1901.05103.

[6] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.

[7] Aristides G Requicha. Representations for rigid solids: Theory, methods, and systems. *ACM Computing Surveys (CSUR)*, 12(4):437–464, 1980.

[8] Starly, Binil. Fabwave -FabWave CAD repository: Categorized part classes.

[9] Cheng Wang, Ming Cheng, Ferdous Sohel, Mohammed Bennamoun, and Jonathan Li. NormalNet: A voxel-based CNN for 3d object classification and retrieval. 323:139–147.

[10] Biao Zhang, Jiapeng Tang, Matthias Niessner, and Peter Wonka. 3dshape2vecset: A 3d shape representation for neural fields and generative diffusion models. _eprint: 2301.11445.